Design Review

Submission Detail	Student Entry
GROUP ID	CSC05
GITHUB REPO LINK	https://github.com/lennoly/INF2004_Grp5

Purpose

Document the system requirements, hardware connections, RTOS task structure, behavioural models, assumptions, and AI-supported design reasoning before major implementation. Treat this Week 6 submission as an architecture freeze before full integration and Tribunal 1. The design may change later, but major changes must be justified with engineering evidence.

Team Role and Subsystem Ownership

Identify each buddy's primary subsystem responsibility. The role does not need to match the examples exactly, but every major hardware and software responsibility should have a clear owner.

Buddy	Student ID	Primary subsystem / responsibility
Buddy 1	2500343	**Communications and telemetry.** Wi-Fi / MQTT connection manager (mqtt_lwip.c on lwIP's client), command parsing and replies, telemetry and heartbeat (command.c, telemetry.c, console_in.c), laptop dashboard.
Buddy 2	2503109	**Motion.** Motor PWM, wheel encoders, per-wheel speed PID with feed-forward, distance and turn moves on a 20 ms control loop, watchdog feed (motor.c, encoder.c, pid.c, motion.c).
Buddy 3	2501660	**Line and barcode.** Three IR sensors on the ADC with a 1 kHz sampler and calibration (ir_sensor.c), line PID and classifier (line_follow.c), Code 39 capture and decoding (barcode.c, barcode_decode.c).
Buddy 4	2503438	**IMU and terrain.** LSM303DLHC over I2C (imu.c): pitch, roll, heading, turn rate, hump height and count (terrain.c), impact detection, motion events.
Buddy 5	2501955	**Obstacles and integration.** HC-SR04 on a servo (ultrasonic.c, servo.c, obstacle.c), obstacle profiling and bypass planning (avoidance.c), the mission state machine (vehicle.c) and integration.

1. Requirements

State what the system must do and the important quality or timing constraints. Requirements should be specific, testable, and traceable to the proposed design (you may add more FR and/or NFR as needed).

1.1 Functional Requirements

ID	Requirement
FR1	Follow the 18 mm black line with two analogue IR sensors and PID steering (state LINE_FOLLOW).
FR2	Confirm line loss after 150 ms with both sensors on white; search with up to five alternating sweeps of growing angle (±k·30°, k = 1…5), starting towards the last-seen side; stop with "line not found" if unsuccessful.

FR3	Read Code 39 barcodes (*A*–*D*) beside the line in either reading direction: A = left, B = right, C = straight at the next junction; D = U-turn at once. Discard and report a junction command not used within 400 mm. Report other valid symbols (e.g. *Z*) without manoeuvring.
FR4	Execute encoder-controlled moves: forward / backward a distance, turn left / right an angle, stop.
FR5	Detect and count humps from IMU pitch; report each peak height and the run maximum.
FR6	Report pitch, roll, heading, turn rate and motion class (stationary, cruising, accelerating, decelerating, turning, climbing, descending, impact); detect impacts from the accelerometer.
FR7	While driving forward, treat an obstacle as ahead after two consecutive valid front readings < 300 mm or an IMU impact; stop, scan (coarse 30–150°, fine ±20° in 5° steps) and estimate the obstacle's edges and side clearances. Each ping is VALID (a range), OUT_OF_RANGE (echo pulse longer than 4 m; treated as free space only if A9 confirms the module gives this pulse when nothing is in range) or FAULT (no echo pulse). Two consecutive FAULTs, or a front reading older than 200 ms while driving forward, stop the car with "sensor fault".
FR8	Bypass on the side needing the smaller offset, or reverse and re-plan (2 attempts shared), or stop with "path blocked"; then rejoin the line. Every forward leg (SIDESTEP, PASS, RETURN, REJOIN) starts only after a 70° / 90° / 110° check shows every reading beyond the leg + 60 mm or OUT_OF_RANGE; the servo is centred and the front monitor active during all forward travel (PASS moves in 50 mm steps, re-centring after each side look). Reverse ≤ 120 mm at ≤ 100 mm/s, only along the path just driven.
FR9	Publish MQTT telemetry at 5 Hz, events when they occur, and a 1 Hz heartbeat with link statistics.
FR10	Accept start, stop, calibrate and tuning commands (speed=, pid=, ff=, line=, rate=) over MQTT, USB or UART. Reply to each with its request id: accepted or rejected (with reason), then applied once the owning task confirms. When queues are full, reject everything except STOP.
FR11	Recover the network without blocking driving: reconnect to the broker after a TCP / MQTT failure (implemented) and re-join the access point after 3 s of link loss, retrying after 5 s, 10 s, then every 20 s (designed, not implemented).
FR12	START (GP20) and STOP (GP21). STOP from the button, USB, UART or MQTT latches, brakes and cancels the current move, scan, calibration and pending commands in every state, and never starts motion. Only an accepted START clears the latch: a short press (acted on release) or "start" begins a run; a ≥ 2 s hold selects calibration. START is accepted only if its health check passes (section 5); otherwise the car stays braked and latched.
FR13	Calibrate with a slow 360° spin (from IDLE by "calibrate" or a long START; from STOPPED by a long START), recording IR white / black levels and magnetometer offsets. Keep the new IR levels only if every sensor's contrast is ≥ 50 % of the ADC range. STOP aborts it.
FR14	Fail safe: brake when a velocity request is older than 100 ms; the motion task feeds a 250 ms RP2040 watchdog only while the vehicle task has made progress within 200 ms; after any reset the car starts braked and never starts a run by itself.

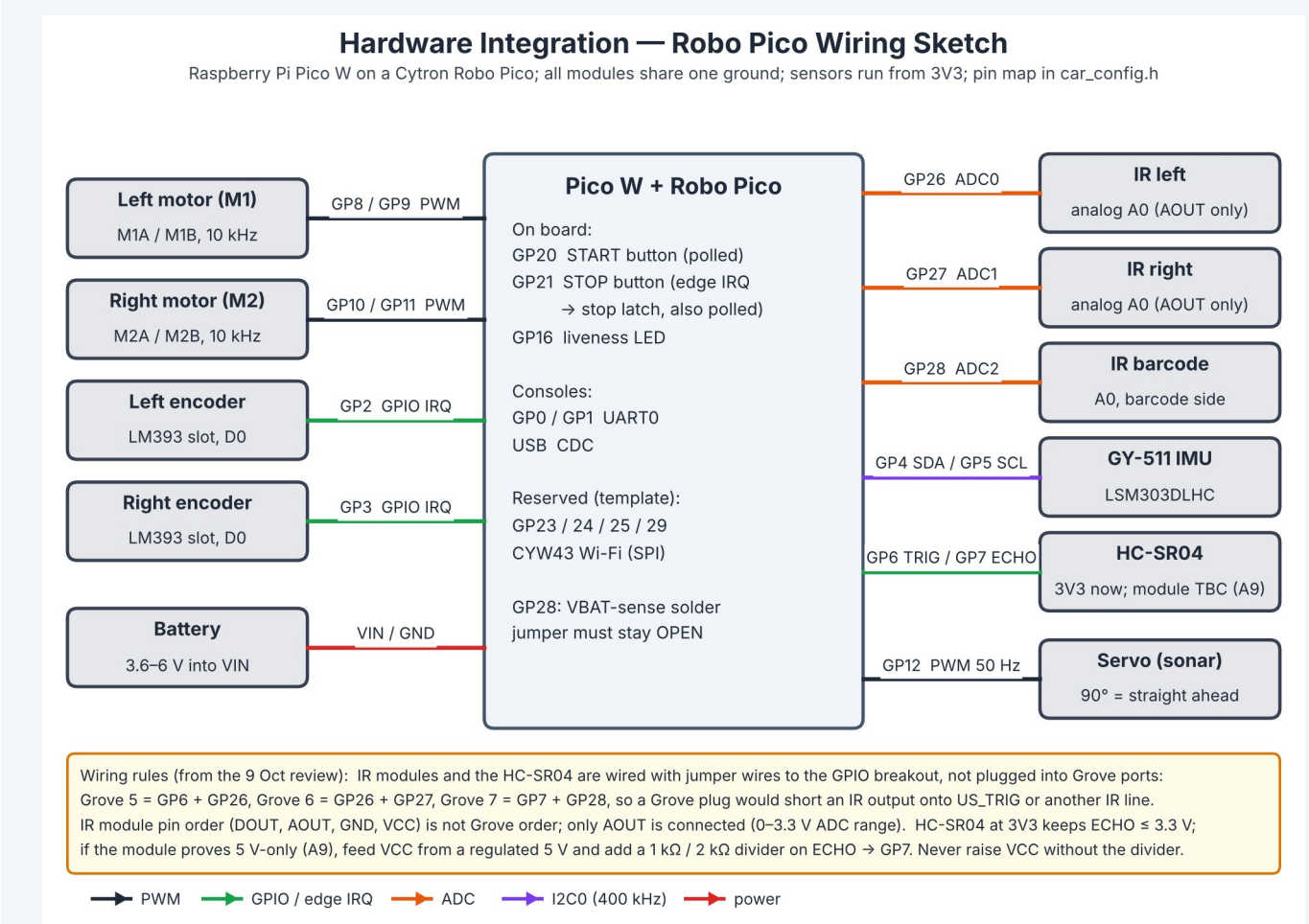
1.2 Non-Functional Requirements

ID	Requirement
NFR1	Wheel-speed control every 20 ms (50 Hz), woken by a kernel cyclic handler; jitter < 2 ms with Wi-Fi telemetry running (A2).
NFR2	IR sampled at 1 kHz by a timer ISR, giving ≥ 10 samples on a 3 mm bar up to 300 mm/s (about 17 at the default 180 mm/s). The limit applies to the barcode sensor, including turns: with sensor offsets x_bc (forward) and y_bc (left), the vehicle task keeps $\sqrt{((v - \omega \cdot y_bc)^2 + (\omega \cdot x_bc)^2)} \le 300$ mm/s, and speed= above 300 mm/s is rejected (the code currently accepts 400 mm/s).
NFR3	STOP brakes the motors within 100 ms of the button edge or of a complete command arriving, in every state, under full load and with command queues full. The GP21 edge IRQ, the USB and MQTT receive paths (before their command rings) and the telemetry task's UART poll set the same latch; motion checks it every 20 ms. Target until measured (A14); an MQTT STOP needs a live connection.
NFR4	Confirm line loss over 150 ms and junctions over 10 mm of travel, so one noisy sample or tape gap cannot trigger an

	action.
NFR5	Front sonar ranges a ≥ 50 mm-wide box from 20 to 400 mm within ±10 mm; a front reading every 70 ms while driving forward with the servo centred (≥ 60 ms between pings, 40 ms echo timeout). "Clear" needs two front readings ≥ 360 mm or OUT_OF_RANGE (A9); a front reading older than 200 ms while driving forward is a sensor fault. Stop budget at 300 mm/s: two readings + vehicle + motion cycles = 180 ms ≈ 54 mm, + braking ≤ 40 mm (A3) + 10 mm error, so the car stops about 190 mm from the box.
NFR6	Read fresh, coherent IMU samples every 20 ms on I2C0 at 400 kHz (10 ms I2C timeout; three stale or failed reads = sensor fault). Hump height within ±5 mm of a ruler on every measured course pass — not yet demonstrated (A7, A12).
NFR7	Straight moves within ±2 % (±10 mm over 500 mm); 90° turns within ±5° on the course surface.
NFR8	Only the cyw43 task calls lwIP. Connection windows 2 / 4 / 8 s; PINGREQ after 30 s idle; session dropped after 45 s with nothing received. A network stall never blocks driving.
NFR9	Every wait is bounded and ends on STOP: encoder 200 ms, echo 40 ms, sonar semaphore 100 ms, move 3 × ideal time + 1 s (≤ 10 s), scan 12 s; vehicle-task waits are sliced ≤ 50 ms. motion_wait() returns OK, FAIL or CANCELLED; only OK may start the next move.
NFR10	C99 and BARR-C:2018: zero warnings under -Wall -Wextra -Wpedantic -Wsign-conversion -Wshadow and 0 tools/barr_check.py findings, deviations documented. Unverified until the reviewed commit, build configuration, compiler log and checker output are attached.
NFR11	Static allocation only (no application malloc). Last linked size ≈ 361 KB flash and 58 KB RAM (.data + .bss, not run-time use); attach the size output for the reviewed commit and keep ≥ 25 % free in every task stack (measured high-water mark).

2. Hardware Integration Pin Layout

Sketch how sensors, actuators, displays, communication modules, and power connections interface with the Robo Pico. Label pins and interfaces such as GPIO, ADC, PWM, I2C, SPI, and UART.



Pin details, interfaces and the RTOS task that owns each connection:

Device / Module	Type	Signal Name	Sensor label	Pico GPIO Pin	Interface	Notes / RTOS Task Owner
Left motor (M1)	Actuator	MOTOR_L_A / MOTOR_L_B	M1A / M1B	GP8 / GP9	PWM (10 kHz)	Owned only by the motion task. A = PWM, B = 0 drives forward; both low brakes (verify on the driver).
Right motor (M2)	Actuator	MOTOR_R_A / MOTOR_R_B	M2A / M2B	GP10 / GP11	PWM (10 kHz)	Owned only by the motion task. MOTOR_x_INVERT in car_config.h fixes a reversed wire.
Left wheel encoder	Sensor	ENCODER_L	D0	GP2	GPIO interrupt	encoder_edge_isr counts both edges (300 µs glitch filter) and time-stamps them; motion reads the counts.
Right wheel encoder	Sensor	ENCODER_R	D0	GP3	GPIO interrupt	As above. Single channel: direction follows the commanded sign (6.2).
IR sensor, left	Sensor	IR_LEFT	A0	GP26	ADC0	Sampled at 1 kHz by ir_sample_isr. AOUT by jumper wire to the GPIO breakout, not Grove 5 / 6 (both carry GP26; Grove 5 also carries GP6 = US_TRIG). Module pin order (DOUT, AOUT, GND, VCC) is not Grove order; DOUT unconnected. 3V3 supply.
IR sensor, right	Sensor	IR_RIGHT	A0	GP27	ADC1	As above; the two sensors straddle the line edges. Record the installed module part (A5).
IR sensor, barcode	Sensor	IR_BARCODE	A0	GP28	ADC2	Mounted 45–50 mm to the barcode side; the ISR times bar / space widths for the barcode task. The Robo Pico VBAT-sense jumper (GP28 = VBAT / 2) must stay open.
GY-511 IMU (LSM303DLHC)	Sensor	IMU_SDA	SDA	GP4	I2C0 (400 kHz)	imu task is the only I2C0 user; the kernel's I2C driver is disabled. Accel 0x19, mag 0x1E.
GY-511 IMU (LSM303DLHC)	Sensor	IMU_SCL	SCL	GP5	I2C0	Clock line of the same bus.
HC-SR04 ultrasonic	Sensor	US_TRIG	TRIG	GP6	GPIO output	10 µs trigger. One semaphore covers each aim + settle + ping (obstacle monitor or vehicle scans).
HC-SR04 ultrasonic	Sensor	US_ECHO	ECHO	GP7	GPIO interrupt	echo_edge_isr times the pulse. Module not identified; wired at 3V3 per the course slide, so ECHO ≤ 3.3 V. The supplied datasheet specifies 5 V, so 3.3 V range is unverified (A9). 5 V fallback: regulated 5 V supply plus 1 kΩ (ECHO → GP7) / 2 kΩ (GP7 → GND) divider; never raise VCC without it.
Servo (sonar mount)	Actuator	SERVO	SIG	GP12	PWM (50 Hz)	90° = straight ahead. Moved only by the task holding the sonar semaphore. V+ = battery voltage (A15).
START / STOP buttons	User input	BTN_START / BTN_STOP	On-board	GP20 / GP21	GPIO input, pull-up; GP21 edge IRQ	START polled every 20 ms (short press on release = start; ≥ 2 s = calibrate). STOP: falling-

						edge IRQ sets the stop latch, so a press is never missed. Press = low.
Liveness LED	Indicator	BOARD_LED	On-board	GP16	GPIO output	Template blink task (priority 10).
Console UART0	Comms	UART0_TX / UART0_RX	TX / RX	GP0 / GP1	UART	Template console; the telemetry task polls the RX FIFO, including STOP.
USB console	Comms	USB CDC	Micro-USB	USB	USB CDC	Template usb task (priority 3); STOP recognised before the console ring.
CYW43439 Wi-Fi (on Pico W)	Comms	WL_ON, WL_D, WL_CS, WL_CLK	Internal	GP23 / 24 / 25 / 29	SPI (PIO)	Template cyw43 task (priority 4) runs lwIP and the MQTT client; MQTT STOP recognised before the command ring. Pins reserved.
Battery	Power	VIN	+ / −	VIN terminal	Power	3.6–6 V into VIN; common ground; sensors on 3V3, motor and servo V+ at battery voltage. Record parts and check stall current against 1 A continuous / 1.5 A (< 5 s) per channel (A15).

3. RTOS Task Table

List the proposed RTOS tasks. Priorities are preliminary at Week 6 and must be justified.

Task	Purpose	Inputs	Outputs	Trigger	Priority
control cyclic handler (kernel)	Wake the motion task on time	Kernel tick (10 ms)	tk_wup_tsk(motion)	Every 20 ms	Handler (above all tasks)
ir_sample_isr (TIMER3 alarm)	Sample the 3 IR channels; time barcode bars and spaces	ADC GP26–28	IR snapshot; width ring (64 entries)	Hardware alarm, 1 kHz	ISR (level 2)
encoder_edge_isr (IO_BANK0)	Count wheel edges and time-stamp them	GP2 / GP3 edges	Edge counts and periods	Each encoder edge	ISR (level 2)
echo_edge_isr (IO_BANK0)	Time the ultrasonic echo pulse	GP7 edges	Echo width; echo event flag	Each echo edge	ISR (level 2)
stop_btn_isr (IO_BANK0)	Latch a STOP press	GP21 falling edge	Stop latch (event flag)	Each STOP press	ISR (level 2)
usb (template)	TinyUSB device and USB console; recognises STOP	USB bus	Console bytes (ring); stop latch	Event / periodic	3
cyw43 (template)	Wi-Fi, lwIP and MQTT client (mqtt_lwip.c); recognises MQTT STOP	Reply queue; TX ring; radio	MQTT packets; command ring; stop latch; link state	Polls every 2 ms	4
motion	Wheel-speed PID, distance and turn moves; brakes on the stop latch; feeds the watchdog	Encoder counts; motion requests; stop latch; vehicle progress	Motor PWM; status; done flag (OK / FAIL / CANCELLED); watchdog feed	Woken every 20 ms	5
vehicle	Mission state machine, line PID, navigation,	IR snapshot; barcode	Motion requests; servo; vehicle	Periodic, 20 ms	6

	obstacle handling	messages; IMU and obstacle status; START button; stop latch; vehicle commands	status; progress		
barcode	Assemble frames and decode Code 39	Width ring	Navigation command (message buffer); stats	Periodic, 10 ms	7
imu	Pitch, heading, humps, impacts, motion events	I2C0 accel / mag; motion status	IMU status snapshot	Periodic, 20 ms	8
obstacle (monitor)	Front range while the servo is centred	HC-SR04 (via semaphore)	Front range, class and time stamp	70 ms while driving forward	9
telemetry	Command executor with replies; telemetry, events, heartbeat; UART poll	Console ring; UART FIFO; command ring; all status snapshots	Vehicle commands; stop latch (UART); replies; TX ring	Periodic, 20 ms	10
blink (template)	Liveness LED	–	GP16	Periodic	10
app_tests (test builds only)	Per-buddy test programs (TEST_MOTOR … TEST_TELEMETRY)	Console	Console logs	Once at start	11

Priority and ownership justification:

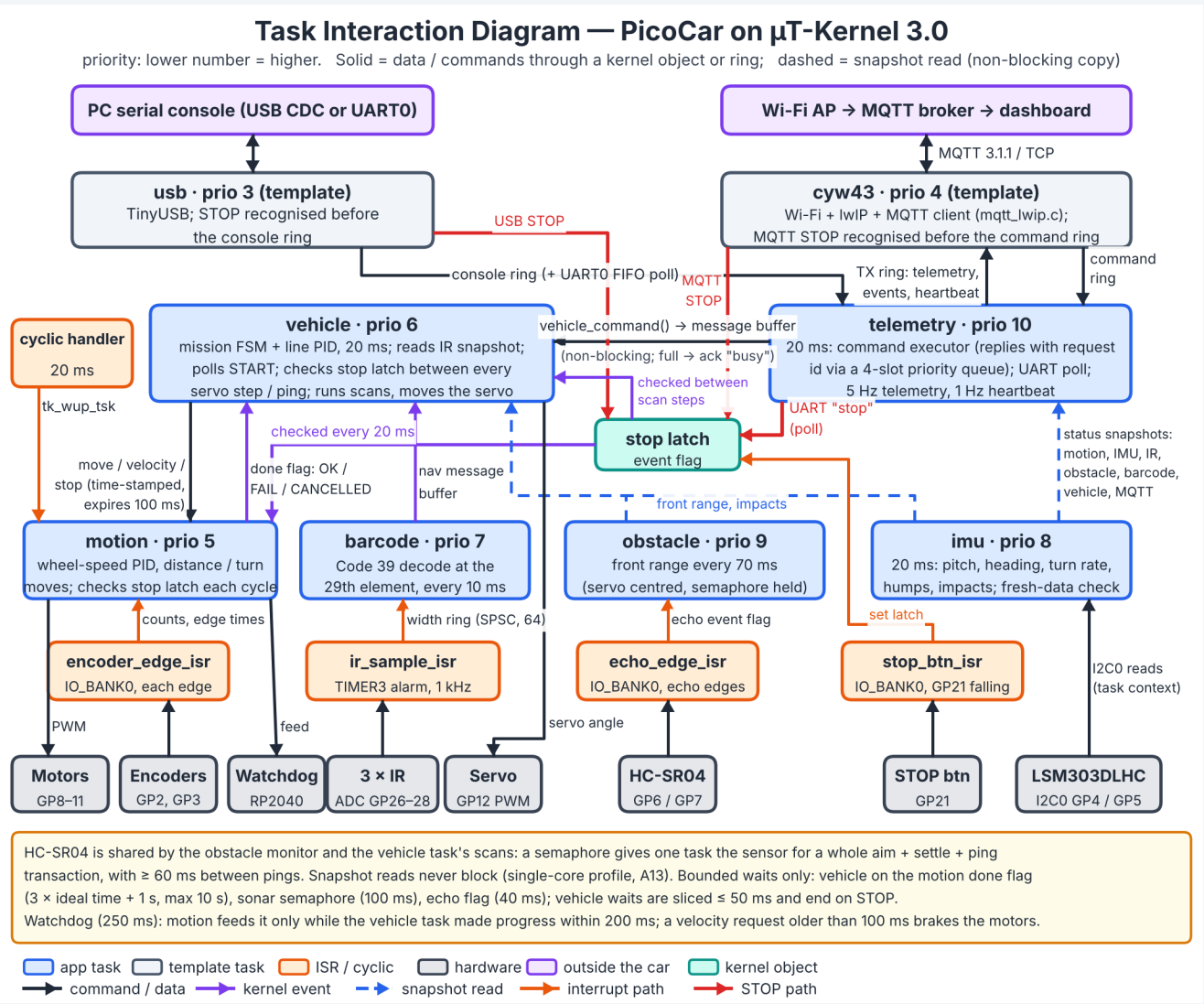
Priorities follow deadlines and consequences (lower number = higher priority in µT-Kernel):

- **ISRs and the cyclic handler** only record data, set the stop latch or wake a task; they never block.
- **motion (5)** has the hardest deadline: a late 20 ms cycle makes the wheels surge, it is the only motor owner, it brakes on the stop latch and it feeds the watchdog.
- **vehicle (6)** makes the 20 ms driving decisions and issues motion requests, so it sits directly below motion.
- **barcode (7)** runs every 10 ms below vehicle: edge timing is done in the ISR and the 64-entry ring holds two full barcodes, so the task can run late without losing data.
- **imu (8)** is soft real-time: hump estimation works on encoder distance, and stale samples are detected by the status bits.
- **obstacle (9)** is limited by the sensor (≥ 60 ms between pings); the 70 ms period meets the NFR5 stop budget.
- **telemetry (10)** is human-facing; its queues drop and count rather than block. STOP does not depend on it except for UART.
- **usb (3) and cyw43 (4)** keep the template priorities their drivers need. They pre-empt motion, so their effect on jitter and STOP latency is measured (A2, A14) and cyw43 moves below motion if needed.

Ownership: one owner per peripheral (motors and watchdog: motion; IR ADC: ir_sample_isr; I2C0: imu; network: cyw43). The sonar and servo are shared by the obstacle monitor and vehicle scans under one semaphore held for each aim + settle + ping.

4. Task Interaction Diagram

Show RTOS tasks, queues, semaphores, mutexes, event groups, notifications, and data flow. Clearly distinguish software tasks from hardware devices and interrupt service routines.



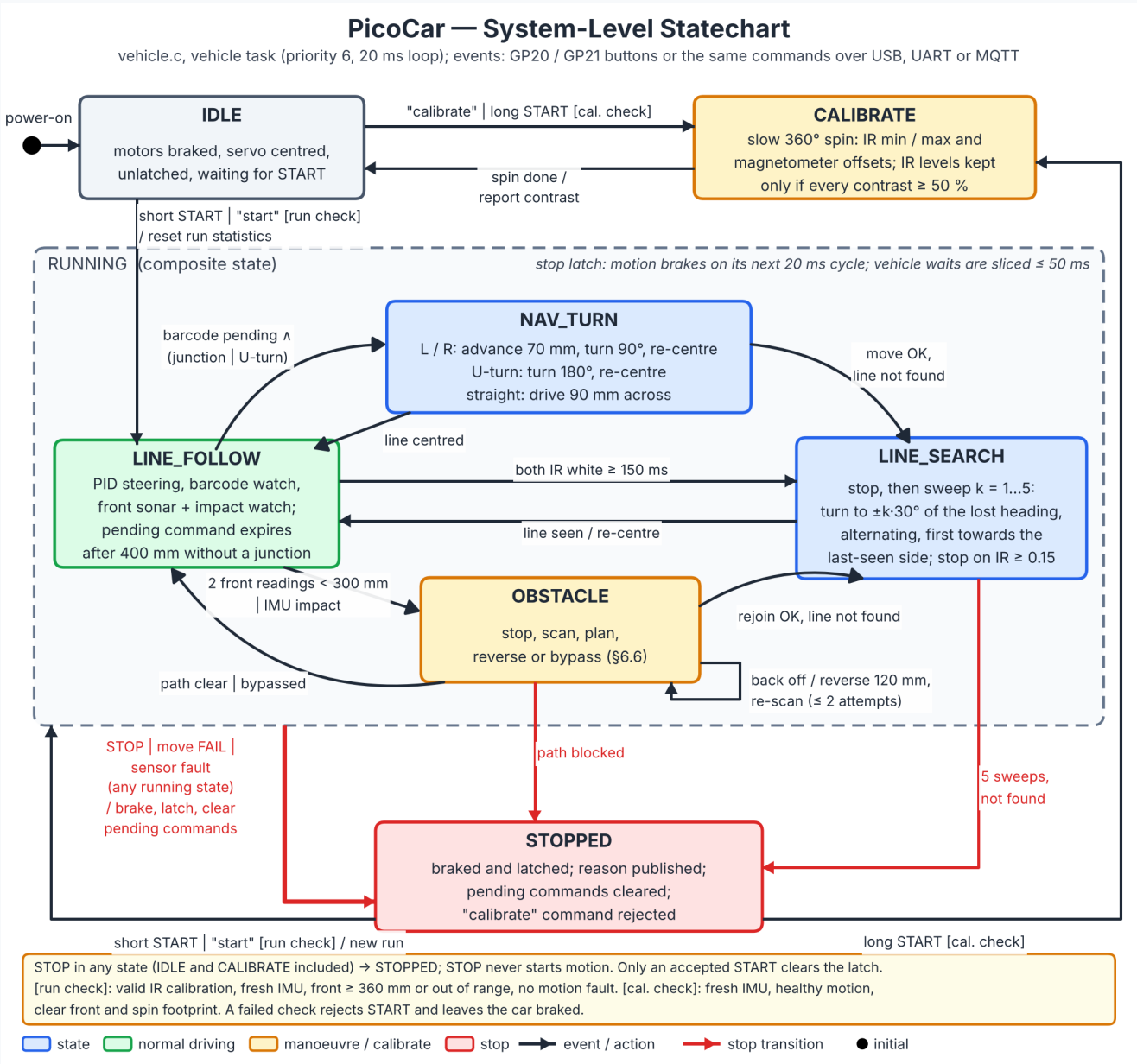
Brief explanation of the main communication paths:

- **Sensor → ISR → task.** ir_sample_isr writes the IR snapshot and pushes bar / space widths into a 64-entry single-producer / single-consumer ring for the barcode task; encoder_edge_isr updates counts for motion; echo_edge_isr sets the echo flag (40 ms timeout); stop_btn_isr sets the stop latch.
- **Control loop.** The 20 ms cyclic handler wakes motion (tk_wup_tsk). The vehicle task posts time-stamped move / velocity / stop requests (newest wins; velocity expires after 100 ms) and waits on the done flag, which returns OK, FAIL or CANCELLED.
- **STOP path.** GP21 IRQ, the usb and cyw43 receive paths (before their rings) and the telemetry UART poll set one latch, independent of any queue. Motion brakes on its next cycle, completes the move as CANCELLED and refuses new ones; the vehicle task (waits sliced ≤ 50 ms) goes to STOPPED and clears pending commands. Only an accepted START clears the latch.
- **Barcode → navigation.** The barcode task sends each decoded command once through a 4-message buffer; the vehicle task polls it without blocking.

- **Snapshots (dashed).** Status structs are copied with interrupts briefly disabled and single words read directly; reads never block (single-core profile, A13). The only blocking waits are the bounded ones in NFR9.
- **Commands and replies.** The telemetry task parses commands and routes them to their owner through a non-blocking message buffer (full → "rejected: busy"). Replies carry the request id and use a 4-slot priority queue sent before telemetry, retried every 500 ms up to 3 times; a repeated id (last 16, kept 30 s after completion) returns the recorded result without re-running. Telemetry uses a 6-slot ring; drops are counted in the heartbeat.
- **Sonar and watchdog.** The sonar semaphore covers each aim + settle + ping and enforces ≥ 60 ms between pings. Motion feeds the 250 ms watchdog only while the vehicle task has made progress within 200 ms.

5. System-Level Statechart

Model the operating states of the entire system. Label transition events or conditions. Include relevant fault, stop, or recovery behaviour.



Explain the main states and any important assumptions:

The vehicle task (vehicle.c, 20 ms) owns these states and checks the buttons, the stop latch and commands in every state.

- **IDLE** — power-on state: braked, servo centred, unlatched. Short START or "start" → LINE_FOLLOW [run check], resetting the run statistics; long START or "calibrate" → CALIBRATE [calibration check].
- **CALIBRATE** — slow 360° spin recording IR min / max and magnetometer offsets; new IR levels kept only if

every sensor's contrast is ≥ 50 %; then IDLE.

- **RUNNING** (composite):
- **LINE_FOLLOW** — PID steering; watches barcodes, the front range and IMU impacts. A pending L / R / straight command expires after 400 mm without a junction.
- **NAV_TURN** — at a junction with a command pending (U-turn at once). L / R: advance 70 mm, turn 90°, re-centre; U-turn: turn 180°, re-centre; straight: drive 90 mm across.
- **LINE_SEARCH** — both sensors white for 150 ms, or a successful move that ends off the line: sweep k = 1 … 5 to ±k·30° of the lost heading, alternating, first towards the last-seen side; IR ≥ 0.15 → re-centre → LINE_FOLLOW; five sweeps without the line → STOPPED "line not found".
- **OBSTACLE** — stop, scan, plan, then bypass, reverse and re-plan, or give up (6.6).
- **STOPPED** — braked and latched; pending commands cleared; reason published (stop command, path blocked, line not found, motion fault, sensor fault). Short START [run check] → new run; long START [calibration check] → CALIBRATE; the "calibrate" command is rejected while latched.

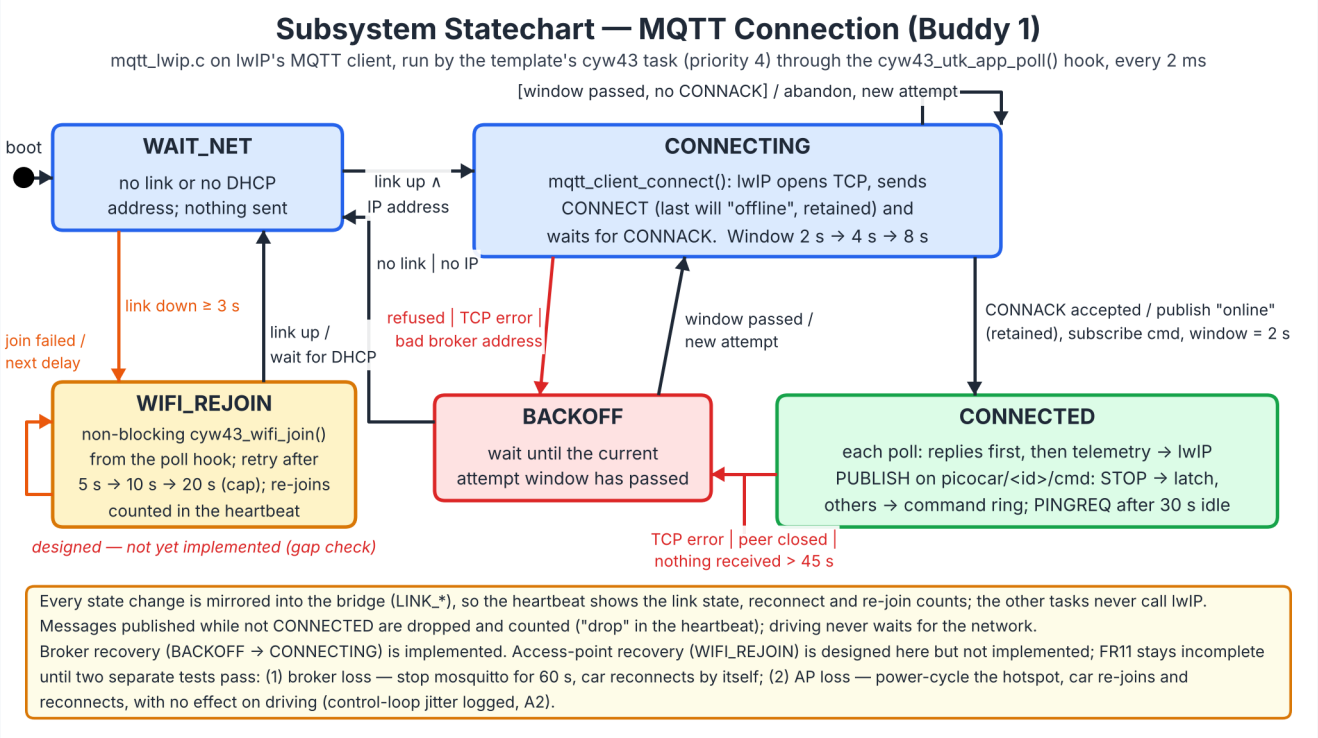
Global: STOP in any state (IDLE and CALIBRATE included), a failed move (FAIL) or a sensor fault → STOPPED; faults never start a line search. [run check]: valid IR calibration, fresh IMU, front reading ≥ 360 mm or OUT_OF_RANGE, no unresolved motion fault. [calibration check]: fresh IMU, healthy motion, clear front and spin footprint (A10). A failed check rejects START with its reason and leaves the car braked and latched; "start" while running is rejected. Assumptions: the car starts on the line; a junction is both sensors black for ≥ 10 mm.

6. Subsystem Statecharts

Add more sub-systems if needed.

6.1 Subsystem Statechart 1 — MQTT Connection (Buddy 1)

Field	Student Entry
Subsystem name	MQTT connection manager (mqtt_lwip.c) on lwIP's MQTT client (lwip/src/apps/mqtt)
Related RTOS task(s)	cyw43 template task (priority 4) through the cyw43_utk_app_poll() hook; telemetry task (priority 10) through the TX / command rings
Purpose	Keep one MQTT 3.1.1 session to the laptop broker, publish replies and telemetry, deliver commands, and recover from broker / TCP failures (implemented) and access-point loss (WIFI_REJOIN, designed) without blocking driving (FR9–FR11, NFR8).
Reason a statechart is needed	The client is event-driven (lwIP callbacks) and has timeouts and retries; a statechart makes each failure path and its recovery explicit so no state can hang and the dashboard can show the link state.



Brief explanation of states, events, and fault handling:

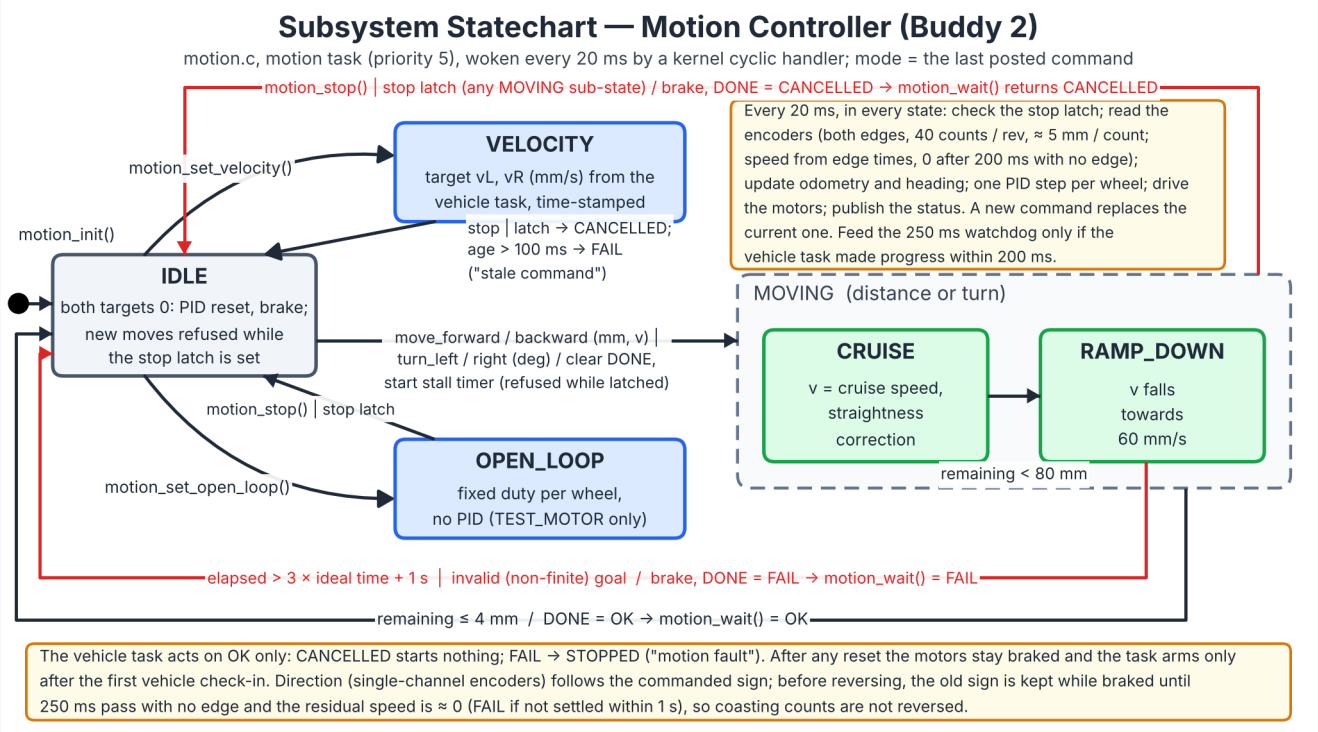
- **WAIT_NET** — no link or no DHCP address; nothing sent. Link + IP → CONNECTING; link down ≥ 3 s → WIFI_REJOIN.
- **WIFI_REJOIN** (designed, not implemented) — non-blocking cyw43_wifi_join() from the poll hook; retry after 5 s, 10 s, then every 20 s; re-joins counted in the heartbeat.
- **CONNECTING** — TCP + CONNECT with retained last will "offline"; attempt windows 2 / 4 / 8 s. CONNACK → CONNECTED: publish "online" (retained), subscribe picocar/<id>/cmd, window back to 2 s.
- **CONNECTED** — each 2 ms poll sends replies first, then telemetry. Incoming STOP sets the latch; other commands go to the command ring. PINGREQ after 30 s idle; session dropped after 45 s with nothing received.
- **BACKOFF** — refused CONNACK, TCP error, peer close or keep-alive timeout: wait for the current window, then retry. Link or IP lost → WAIT_NET.

State and recovery counts appear in the heartbeat; messages published while disconnected are dropped and counted, and replies that never got through are reported as unconfirmed. FR11 stays incomplete until broker loss (stop mosquitto 60 s) and AP loss (power-cycle the hotspot) each pass separately.

6.2 Subsystem Statechart 2 — Motion Controller (Buddy 2)

Field	Student Entry
Subsystem name	Motion controller (motion.c with pid.c, motor.c, encoder.c)
Related RTOS task(s)	motion task (priority 5), woken every 20 ms by a kernel cyclic handler; requests come from the vehicle task (and the test programs)
Purpose	Turn distance, angle and speed requests into wheel PWM (per-wheel PID with feed-forward), end every move as OK, FAIL or CANCELLED, brake on the stop latch or a stale request, and feed the watchdog (FR4, FR14, NFR1, NFR7, NFR9).
Reason a statechart is needed	The controller behaves differently by mode (idle, velocity, open loop, moving with ramp-

	down) and must end every move in a defined way (done, stopped or failed); a statechart shows each mode, the completion rules and the stall timeout.



Brief explanation of states, events, and fault handling:

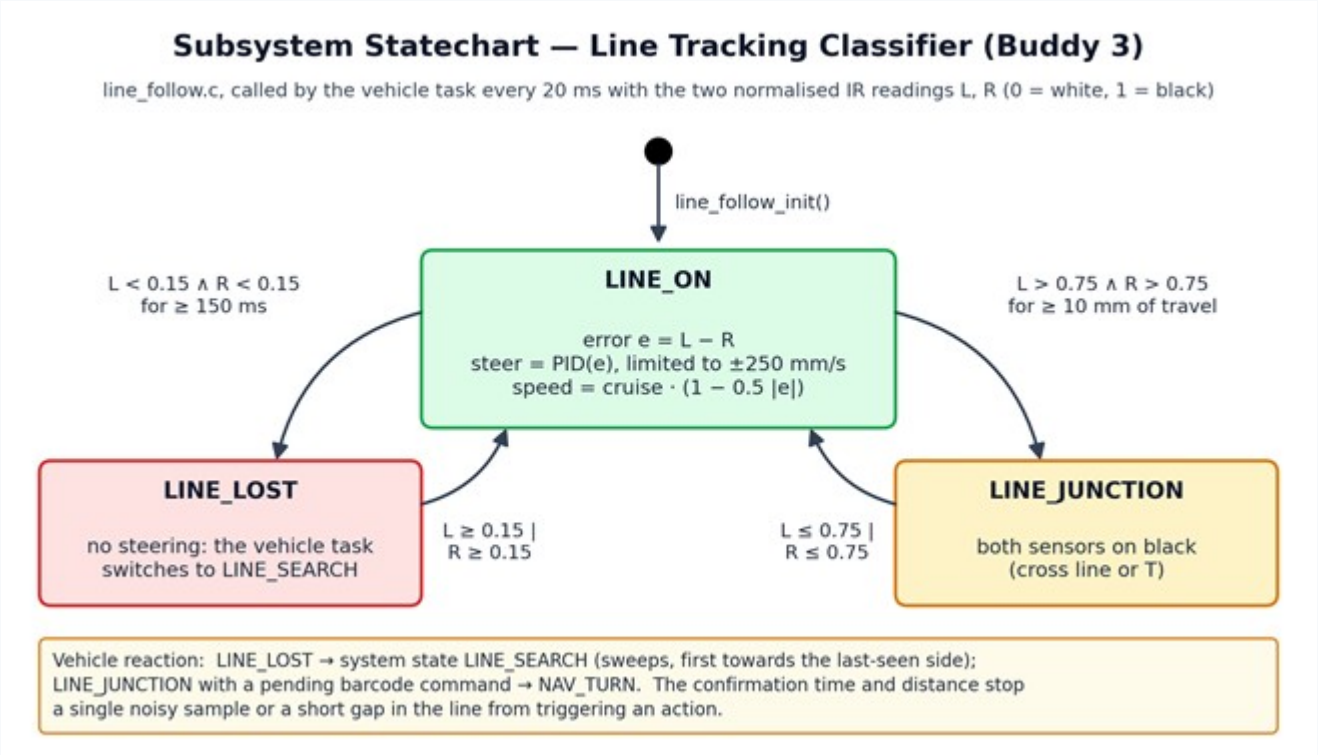
- **IDLE** — targets 0, PID reset, braked; new moves refused while latched. After any reset the task stays here, braked, and arms only after the first vehicle check-in.
- **VELOCITY** — time-stamped wheel speeds from the vehicle task; older than 100 ms → brake, FAIL "stale command". **OPEN_LOOP** — fixed duty, TEST_MOTOR only.
- **MOVING** — **CRUISE** at the requested speed with straightness correction; **RAMP_DOWN** when < 80 mm (or equivalent arc) remains, towards 60 mm/s.
- **Results** — remaining ≤ 4 mm: OK. Timeout (3 × ideal + 1 s) or non-finite goal: brake, FAIL → vehicle goes to STOPPED "motion fault". motion_stop() or the stop latch: brake, CANCELLED → the vehicle starts nothing.

Every cycle: check the latch, read the encoders (both edges of the 20-slot disc, 40 counts / rev ≈ 5 mm per count; speed from edge times, 0 after 200 ms without an edge), update odometry and heading, run the PIDs, publish status, feed the watchdog only if vehicle progress is < 200 ms old. Single-channel encoders: direction follows the commanded sign; before reversing, the old sign is kept while braked until 250 ms pass with no edge and residual speed ≈ 0 (FAIL if not settled within 1 s).

6.3 Subsystem Statechart 3 — Line Tracking Classifier (Buddy 3)

Field	Student Entry
Subsystem name	Line tracking classifier (line_follow.c)
Related RTOS task(s)	vehicle task (priority 6), every 20 ms; IR data from ir_sample_isr (1 kHz)
Purpose	Classify the two normalised IR readings as on the line, line lost or junction, and compute the PID steering and speed while on the line (FR1, FR2, NFR4).

Reason a statechart is needed	Line loss and junctions must be confirmed over time or distance, not from one sample; the statechart makes these confirmation rules and the recovery back to LINE_ON explicit.



Brief explanation of states, events, and fault handling:

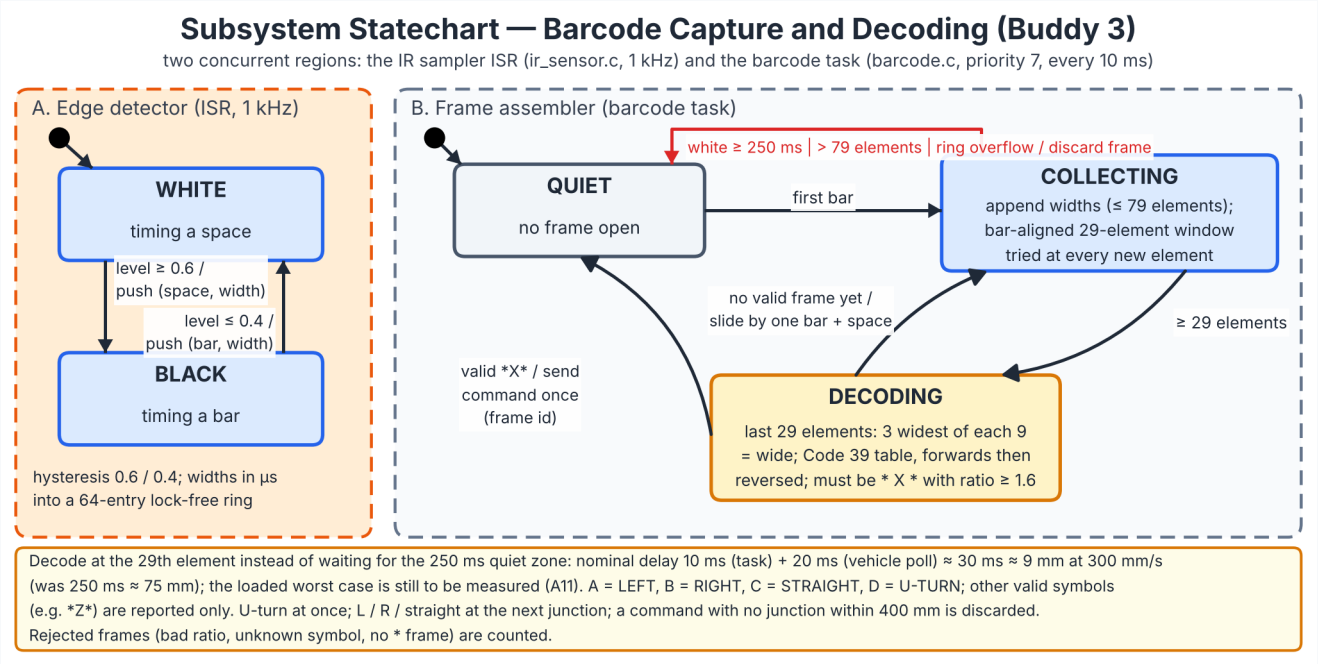
- **LINE_ON** — normal tracking: error e = L − R (0 = white, 1 = black), steering = PID(e) limited to ±250 mm/s, forward speed = cruise × (1 − 0.5 |e|) so the car slows in bends.
- **LINE_LOST** — both readings below 0.15 for at least 150 ms. The classifier gives no steering here: the vehicle task stops the car and searches (LINE_SEARCH), sweeping first towards the side the line was last seen. Any reading of 0.15 or more returns to LINE_ON.
- **LINE_JUNCTION** — both readings above 0.75 for at least 10 mm of travel (a cross line or T). Either reading dropping to 0.75 or below returns to LINE_ON.

The vehicle task reacts to the classifier: LINE_LOST → system state LINE_SEARCH; LINE_JUNCTION with a pending barcode command → NAV_TURN. The time and distance thresholds stop a noisy sample or a short gap in the tape from causing an action. Calibration (CALIBRATE state) sets the white / black levels used for normalising.

6.4 Subsystem Statechart 4 — Barcode Capture and Decoding (Buddy 3)

Field	Student Entry
Subsystem name	Barcode capture and decoding (ir_sensor.c edge detector, barcode.c, barcode_decode.c)
Related RTOS task(s)	ir_sample_isr (1 kHz, region A); barcode task (priority 7, every 10 ms, region B); result to the vehicle task through a message buffer
Purpose	Measure bar and space widths from the barcode IR sensor, assemble a frame, decode *X* in either reading direction and send the navigation command (FR3, NFR2).

Reason a statechart is needed	Capture (ISR) and decoding (task) run concurrently; the statechart shows both regions, the frame rules (bar-aligned 29-element window, 250 ms reset) and every rejection path.

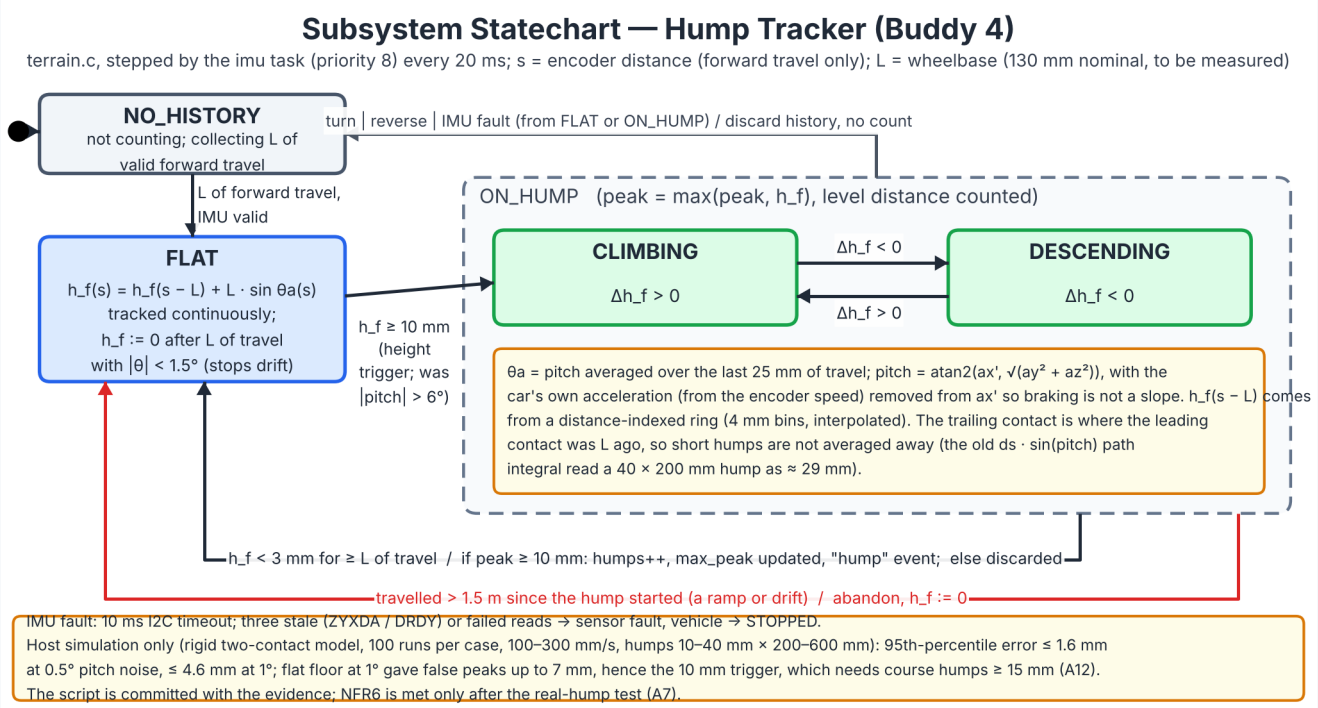


Brief explanation of states, events, and fault handling:

- **Region A (ISR)** — WHITE / BLACK with 0.6 / 0.4 hysteresis; each change pushes the element and its width (µs) into the 64-entry lock-free ring. Ring overflow invalidates the current frame.
- **Region B (task)** — QUIET until the first bar, then COLLECTING (≤ 79 elements). From the 29th element, each new element triggers DECODING of the last 29, starting and ending on a bar; an invalid window slides by one bar + space. White ≥ 250 ms, a runaway or an overflow discards the frame.
- **DECODING** — the 3 widest of each 9 elements are wide; Code 39 lookup forwards, then reversed; must be *X* with wide / narrow ratio ≥ 1.6. A–D are sent once per frame (frame id); other valid symbols are reported only. Bad ratio, unknown symbol or missing * are counted.
- **Timing** — decoding without the quiet zone gives a nominal 10 ms (task) + 20 ms (vehicle poll) ≈ 30 ms ≈ 9 mm at 300 mm/s, against 250 ms ≈ 75 mm before. The loaded worst case is still to be measured (A11).

6.5 Subsystem Statechart 5 — Hump Tracker (Buddy 4)

Field	Student Entry
Subsystem name	Hump tracker (terrain.c)
Related RTOS task(s)	imu task (priority 8), every 20 ms; distance from the motion status (encoders); hump events published by the telemetry task
Purpose	Estimate the leading-contact height from pitch and encoder distance, detect and count humps and report the peak (FR5, NFR6).
Reason a statechart is needed	Detection, peak and end are decided from noisy pitch over distance, and the height history must be valid; the statechart fixes the trigger, end, drift reset, abandon and history-invalid rules.



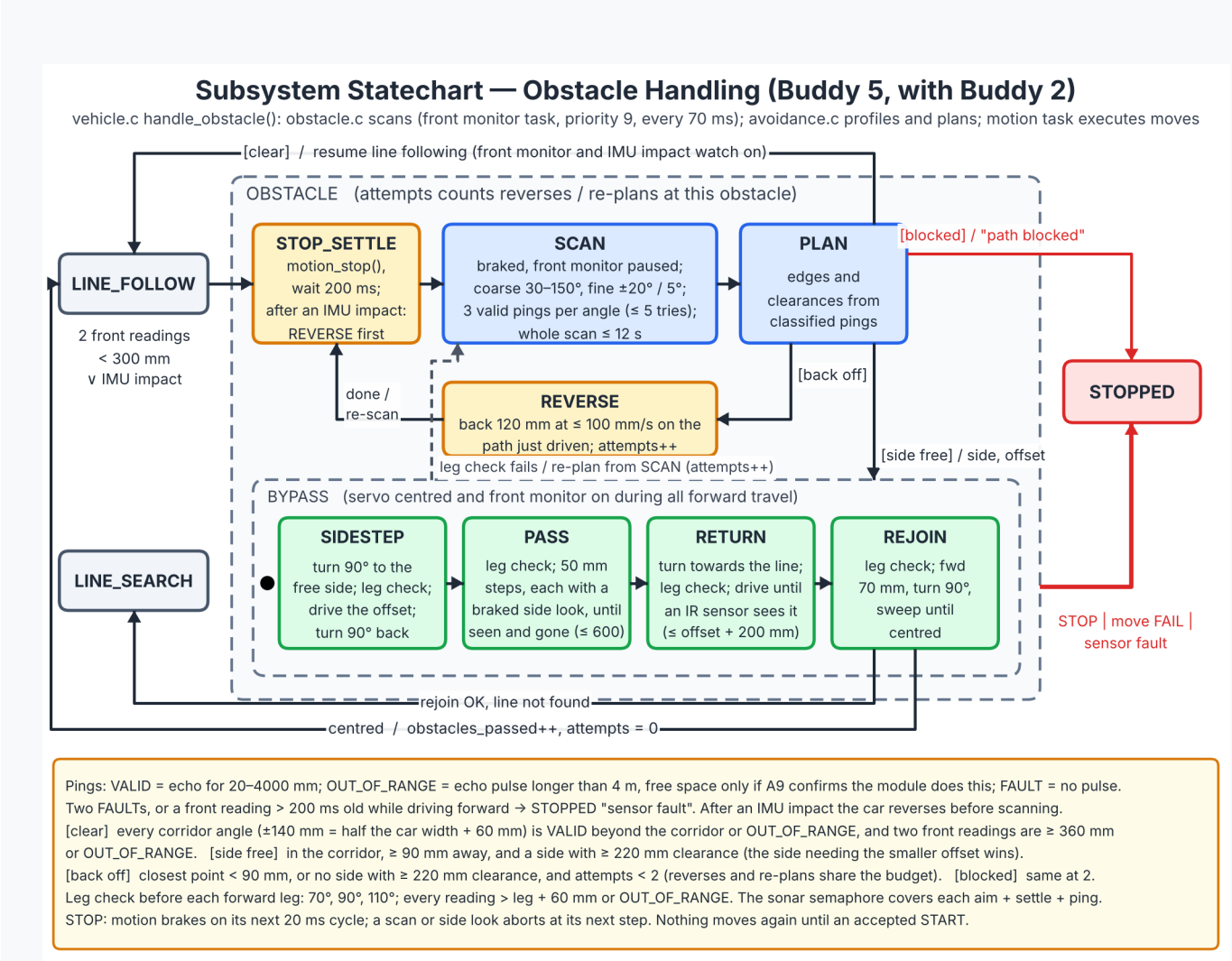
Brief explanation of states, events, and fault handling:

- **Estimator** — body pitch θ is the slope between the trailing and leading contacts, a wheelbase L apart (130 mm nominal, to be measured), so h_f(s) = h_f(s − L) + L · sin θa(s), where s is encoder distance and θa is pitch averaged over the last 25 mm. h_f(s − L) comes from a 4 mm-bin distance ring with interpolation; the car's own acceleration is removed before computing pitch. It replaces the ds · sin(pitch) integral, which read a 40 × 200 mm hump as about 29 mm.
- **NO_HISTORY → FLAT** — counting starts only after L of valid forward travel. A turn, reverse or IMU fault discards the history and any candidate.
- **FLAT** — h_f tracked; reset to 0 after L of travel with |θ| < 1.5°. **ON_HUMP** — h_f ≥ 10 mm (height trigger above the 7 mm simulated flat-floor noise; detects humps ≥ 15 mm with margin, A12; the old 6° pitch trigger missed a 20 × 600 mm hump peaking at 5.98°); peak = max h_f. **End** — h_f < 3 mm for L: count it and publish. **Abandon** — > 1.5 m on one hump.
- **IMU set-up** — accelerometer 100 Hz, ±2 g, high-resolution, block data update; magnetometer 75 Hz, ±1.3 gauss. ZYXDA / DRDY checked each read; three stale or failed reads → sensor fault → STOPPED. Turn rate from the encoder differential, cross-checked with the wrapped (±180°) magnetometer heading change, which is ignored when |B| leaves ±25 % of calibration (A16).
- **Evidence** — host simulation only (rigid two-contact model, 100 runs per case, 100–300 mm/s, humps 10–40 mm × 200–600 mm): 95th-percentile error ≤ 1.6 mm at 0.5° pitch noise and ≤ 4.6 mm at 1°; flat floor at 1° gave false peaks up to 7 mm, hence the 10 mm trigger. The script is committed with the evidence; NFR6 is met only after the real-hump test (A7).

6.6 Subsystem Statechart 6 — Obstacle Handling (Buddy 5, with Buddy 2)

Field	Student Entry
Subsystem name	Obstacle handling (vehicle.c handle_obstacle, obstacle.c, avoidance.c, ultrasonic.c, servo.c)
Related RTOS task(s)	vehicle task (priority 6) runs the sequence; obstacle task (priority 9) monitors the front range; motion task (priority 5) executes the moves

Purpose	Stop before an obstacle, profile it, choose bypass / reverse / stop, pass it with the smaller offset and rejoin the line, never driving on unchecked or faulty sensing (FR7, FR8, NFR5).
Reason a statechart is needed	A multi-step sequence with guards, a shared retry budget and several ways to fail; the statechart shows each decision and where each failure leads (STOPPED, or LINE_SEARCH only after a successful rejoin that misses the line).



Brief explanation of states, events, and fault handling:

- **Ping classes** — VALID (20–4000 mm), OUT_OF_RANGE (echo pulse longer than 4 m: counted as free space only if A9 confirms the module gives this pulse when nothing is in range), FAULT (no echo pulse). Two FAULTs in a row, or a front reading older than 200 ms while driving forward → STOPPED "sensor fault".
- **STOP_SETTLE → SCAN → PLAN** — brake, wait 200 ms (after an IMU impact, reverse 120 mm first: a touching box is inside the 20 mm blind zone). SCAN runs braked with the front monitor paused: coarse 30–150°, fine ±20° in 5° steps, 3 valid pings per angle in ≤ 5 tries, whole scan ≤ 12 s. avoidance.c turns the points into edges and side clearances.
- **[clear]** — every corridor angle (±140 mm = half the car width + 60 mm) is VALID beyond the corridor or OUT_OF_RANGE, and two front readings are ≥ 360 mm or OUT_OF_RANGE → LINE_FOLLOW.
- **[back off]** — closest < 90 mm or no side with ≥ 220 mm: REVERSE 120 mm at ≤ 100 mm/s along the path just driven, then re-scan; reverses and re-plans share 2 attempts, then **[blocked]** → STOPPED "path blocked".
- **[side free] → BYPASS** — SIDESTEP (turn 90°, leg check, drive the offset, turn back), PASS (leg check; 50 mm

steps, each followed by a braked side look and re-centring, until the obstacle is seen and gone, ≤ 600 mm, then one car length), RETURN (turn, leg check, drive until an IR sensor sees the line, ≤ offset + 200 mm), REJOIN (leg check, forward 70 mm, turn 90°, sweep until centred) → LINE_FOLLOW. Leg check: 70°, 90°, 110° all beyond leg + 60 mm or OUT_OF_RANGE, else re-plan (uses an attempt). The servo is centred and the front monitor active during all forward travel.

- **Faults** — STOP, a failed move or a sensor fault → STOPPED. Only a successful rejoin that ends off the line leads to LINE_SEARCH. Corners closer than about 150 mm outside the leg-check cone, and the swept area of turns, are not covered yet; validation planned (A10).

7. Assumptions

List assumptions that the design depends on. Each assumption must state the risk if it is wrong and how the team will validate it before full integration. Have **at least** 1 assumption per buddy.

ID	Assumption	Risk if wrong	Validation method	Owner	Status
A1	The lab access point and laptop broker (mosquitto, port 1883) are reachable from the car with no client isolation and < 100 ms round trip.	No telemetry or remote commands in the demo.	TEST_TELEMETRY for 10 min on the car in the lab; check heartbeat reconnects and drops. So far only against a local broker with sim_car.py.	Buddy 1	Partially validated (host only)
A2	The template cyw43 (4) and usb (3) tasks delay the 20 ms control loop by < 2 ms.	Wheel surging, distance overshoot, longer STOP latency.	Time-stamp every control cycle with 5 Hz telemetry, during a broker reconnect and an AP re-join; if jitter > 2 ms, move cyw43 below motion and re-test.	Buddy 1	Open
A3	Both encoder edges are clean (≥ 300 µs apart) up to 300 mm/s, giving 40 counts / rev (≈ 5 mm per count); slip is small; braking from 300 mm/s takes ≤ 40 mm.	Motion errors (NFR7), false stall timeouts, longer stop distance than NFR5 assumes.	Count edges over 10 hand-turned revolutions; TEST_MOTION: 5 × 500 mm runs and 5 × 90° turns each way; log speed at 30–60 mm/s, coast after braking from 180 and 300 mm/s, and counts on a reverse right after a forward move.	Buddy 2	Open
A4	The motors respond roughly linearly above an 18 % dead-band, and battery sag over a run is small enough for PID + feed-forward.	Stalls at low speed or overshoot on a low battery.	TEST_MOTOR: open-loop PWM sweep and PID step response at full and low battery; tune kF and the offset with live tuning.	Buddy 2	Open
A5	Every IR module gives ≥ 50 % ADC contrast at its mounting height under lab lighting.	Missed lines, junctions or bars; calibration keeps rejecting.	Record the module part; TEST_IR min / max on the course under normal and window lighting; check each sensor sees black and white during the spin.	Buddy 3	Open
A6	Barcode-sensor speed stays ≤ 300 mm/s and within ±20 % over one 141 mm barcode.	Wrong or missed commands.	TEST_BARCODE: 20 passes at 100, 180 and 300 mm/s, both directions and on a bend; ≥ 90 % correct, 0 wrong. Measure x_bc, y_bc.	Buddy 3	Partially validated (synthetic vectors)
A7	Averaged pitch noise ≤ 1°, wheelbase L known to ±1 mm, and the two-contact model fits the chassis.	False humps or height error > 5 mm.	Measure L; 5 m flat at each speed: no hump counted (h_f never ≥ 10 mm); 5 passes per course hump per speed, every error ≤ 5 mm. Commit the simulation script with the logs.	Buddy 4	Partially validated (host simulation)

A8	An impact gives a sample-to-sample change > 700 mg that normal driving (bumps, braking, humps) never gives.	Missed collisions or false obstacle stops.	Compare the largest jerk over normal laps and humps with gentle pushes into a box; set IMPACT_JERK_MG between them.	Buddy 4	Open
A9	The installed ultrasonic module ranges a 50 mm box from 20 to 400 mm at 3.3 V, and with nothing in range gives a recognisable long echo pulse (OUT_OF_RANGE) rather than none; servo 0° points to the car's right.	Missed obstacles; if the module gives no pulse at all with nothing in range, open track looks like FAULT and the car stops.	Identify the module; at 3.3 V ping boxes at 20, 150, 300, 360 and 400 mm, straight and offset (20 pings each, ≥ 95 % valid, ±10 mm); ping open space and log the pulse. If range fails, use the 5 V + divider fallback. If there is no pulse with nothing in range, no-echo cannot be told apart from a fault, and a different clearance method is needed before integration. Check servo direction.	Buddy 5	Open
A10	There is ≥ 220 mm beside each obstacle, the line continues past it, and the path just driven, turn sweeps and calibration spin are clear.	Collision or failed rejoin.	5 bypasses per course placement; record minimum clearances, near-corner contacts, turn sweeps and rejoins; confirm each 120 mm retreat path and the calibration spin area.	Buddy 5	Open
A11	Every barcode is on the sensor's side for each direction it is read (the side flips after a U-turn), and its junction starts 30–400 mm after it.	Unreadable barcode; command too late or expired.	Check the course layout for every travel direction; log decode-to-junction distance at 180 and 300 mm/s under load. If a barcode must be read from the other side, add a module there using its DOUT on a free GPIO interrupt.	Buddy 3	Open
A12	Every course hump is ≥ 15 mm high (10 mm trigger + 5 mm for the ≤ 4.6 mm simulated estimate error), and the sonar does not see a hump as an obstacle.	A low hump that reads low is never counted; humps treated as obstacles.	Measure the smallest course hump with a ruler; TEST_IMU over it at every speed, logging h_f, pitch and front range. If it is < 15 mm, lower the trigger only if the flat-floor logs allow (A7). Aim the sonar higher if the hump echoes.	Buddy 4	Open
A13	The firmware runs on the single-core µT-Kernel profile, so interrupt-protected snapshot copies are atomic.	Torn snapshots on a dual-core (SMP) build.	Check the profile of every build; add a compile-time #error for SMP.	Buddy 5	Open
A14	Under full load STOP reaches the motors within 100 ms in every state, and a stalled task stops the car within 500 ms.	The car keeps moving in exactly the busy states where STOP matters.	20 STOP injections each from button, USB, UART and MQTT in every state (scans, side looks, NAV_TURN, CALIBRATE) with telemetry streaming and queues flooded; logic analyser from GP21 edge / command arrival to motor pins low. Test START accept / reject, short / long START, calibration abort. Suspend vehicle, then motion: motors stop ≤ 500 ms; reset starts braked.	Buddy 5	Open
A15	The motors draw ≤ 1 A continuous and ≤ 1.5 A for < 5 s per channel, and the servo is rated for the battery voltage.	Driver overload, brown-out resets, servo damage.	Record motor, servo and battery parts; measure start and stall current at full battery; check the servo rating.	Buddy 2	Open
A16	With the motors running, the magnetometer field	Wrong heading and turn rate;	Log |B|, heading and both turn-rate estimates over laps and turns;	Buddy 4	Open

	stays within ±25 % of its calibrated magnitude on the course.	misclassified motion events.	check axis mapping, wrap-around and freshness.		

8. AI-Supported Design Evidence

Document how AI was used to support the design. The aim is not to show that AI produced perfect answers, but to show that the team used AI critically, corrected weak suggestions, and made final engineering decisions.

8.1 Pre-Submission AI Ideation

Buddy	AI prompt or discussion used	Useful AI ideas	Weak or unrealistic AI ideas	Team decision
Buddy 1	“The µT-Kernel template's cyw43 task owns lwIP (NO_SYS). Design an MQTT client so the other tasks can publish telemetry and receive commands without calling lwIP.”	Run the client inside the cyw43 task through a weak poll hook; exchange data through single-producer / single-consumer rings; last will “offline” plus a retained “online”; exponential back-off.	The first version sent PINGREQ only when the link was idle. With 5 Hz telemetry it never pinged, so a dead broker was never noticed. It also reported new gains before the vehicle task had applied them.	PINGREQ on a fixed schedule plus a 45 s receive limit; later replaced by lwIP's own client, whose watchdog also counts TCP acknowledgements (NFR8, 6.1). The fixed-delay "applied" report was replaced by request ids with owner-confirmed replies (FR10).
Buddy 2	“Structure a 50 Hz per-wheel speed PID and distance / turn moves on µT-Kernel 3.0. How should the loop be timed and how does a caller wait for a move?”	A kernel cyclic handler that only wakes the task; feed-forward + static-friction offset; ramp-down before the target; a stall timeout; an event flag for “done”.	The TEST_MOTOR program created a second motion task every time it was re-run, and its open-loop PWM sweep fought the PID.	Initialisation made idempotent; an OPEN_LOOP mode added for measuring the motors; moves fail after 3 × ideal time + 1 s so no caller waits forever.
Buddy 3	“How can one analog IR sensor read Code 39 barcodes at an unknown speed and in either direction?”	Time bar / space widths instead of sampling positions; call the 3 widest of each 9 elements wide; try the frame forwards, then reversed.	Sampling in a 10 ms task would give about one sample per 3 mm bar at cruise speed. One AI-written test vector did not match the Code 39 table.	Sampling moved to a 1 kHz hardware-alarm ISR (NFR2) with 0.6 / 0.4 hysteresis; every table entry and test vector checked against the course barcode specification and the sample photo (*A*).
Buddy 4	“The GY-511 has no gyroscope. How can we estimate hump height and detect collisions with only an accelerometer, a magnetometer and wheel encoders?”	Integrate sin(pitch) × encoder distance instead of double-integrating acceleration; subtract the car's own acceleration (from encoder speed) before computing pitch; impact from sample-to-sample jerk.	Double integration of acceleration was also suggested (it drifts within seconds). The first hump tracker read about 14 % low in host simulation because the climb below the 6° trigger was lost.	Path integral kept at first with a pre-roll height; after review replaced by the front-contact estimator with a 10 mm height trigger (6.5), because body pitch averages short humps away.
Buddy 5	“Plan obstacle avoidance with an ultrasonic sensor on a servo, then review the whole design for consistency and coding-standard compliance.”	Coarse-then-fine scan; a corridor test for “in the way”; pick the side needing the smaller offset; a box-shaped bypass that watches the obstacle with a side-looking sonar.	The profile merged an obstacle with a wall beside it, so both sides looked blocked. The AI also stated the code “follows Barr-C” when our checker found 1,570 violations.	Points clustered by depth so neighbours are separated from the obstacle. We wrote tools/barr_check.py and worked towards 0 findings with documented deviations; output for the reviewed commit to be attached (NFR10).

8.2 Team-Reviewed Architecture Freeze

AI or initial idea	Team decision	Engineering reason	Affected artefact
One RTOS task per subsystem, priorities by deadline (motion 5, vehicle 6, barcode 7, imu 8, obstacle 9, telemetry 10).	Accepted	Matches buddy ownership so each task can be tested alone (TEST_* builds). Motion gets the highest application priority; the template usb and cyw43 tasks still pre-empt it, so timing is measured (A2, A14).	RTOS task table, task interaction diagram, team roles
Create our own network task for MQTT.	Modified	lwIP runs in NO_SYS mode and is not thread-safe, and the template's cyw43 task already owns it. The client runs inside that task through a weak hook; other tasks use rings only.	Task table, task interaction diagram, MQTT statechart, NFR8
Use the kernel's ADC and I2C device drivers.	Rejected with reason	They use blocking device-manager calls that cannot run in a 1 kHz ISR. Disabled in config_device.h; each peripheral has one owner (IR ADC: ISR, I2C0: imu task).	Pin layout, task table, assumptions A5 / A7
Sample the IR sensors from a 10 ms task or the kernel tick.	Rejected with reason	A 3 mm bar lasts about 17 ms at cruise speed; 10 ms sampling gives one or two samples. A TIMER3 alarm ISR at 1 kHz gives about 17.	NFR2, barcode statechart, task table
Use the newest Pico SDK.	Rejected with reason	The template fails to build with newer SDKs (fixed_bitset.h). SDK pinned at 2.2.0 and recorded in the build notes.	Build set-up, integration plan
Hump height by double integration of acceleration.	Rejected with reason	Drift grows with time squared. The front-contact estimator only drifts with pitch error and resets on flat floor.	Hump statechart, FR5, NFR6, A7
Read console commands with the template's tm_getchar().	Rejected with reason	It spins with interrupts disabled, which would stall the control loop. A non-blocking ring filled by a USB receive hook, plus a UART FIFO poll, is used instead.	Task interaction diagram, FR10
Use floating point in the control code (BARR-C 5.4.a prefers fixed width types).	Accepted with deviation	PID and geometry are far clearer in float; a float32_t typedef and isfinite() guards are used, and the deviation is documented. The CPU load (RP2040 has no FPU) must be measured with A2.	NFR10, coding-standard deviation list
STOP in IDLE starts calibration; STOP button only polled.	Rejected with reason	STOP must never move the car, and a press during a 2.5–11 s scan was missed. STOP is now a latch set by an edge IRQ and receive paths; calibration uses "calibrate" or a long START.	FR12, FR13, sections 4–5, A14
Hump height from ds · sin(pitch) with a 6° trigger.	Modified	It under-read short humps (~29 mm for 40 mm) and missed a 20 × 600 mm hump. Front-contact estimator with a 10 mm height trigger.	6.5, NFR6, A7, A12
Decode a barcode after the 250 ms quiet zone; turn after 400 mm without a junction.	Modified	At 300 mm/s the command arrived 75 mm late. Decoding at the 29th element gives ≈ 9 mm; an unused command is now discarded instead of turning off the line.	FR3, 6.4, A11
Treat no echo as "unknown" and stop after two.	Modified	On open track the HC-SR04 sees nothing within 4 m, so the car would stop or refuse START. Out-of-range is free space if A9 confirms the module behaviour; a missing echo pulse or a stale reading is a fault.	FR7, FR8, NFR5, 6.6, A9
Report gains "applied" after a fixed 60 ms delay.	Rejected with reason	A delay is not a confirmation. Replies carry request ids and the owner's real outcome.	FR10, section 4

8.3 Final AI-Vetted Gap Check

After completing the design, ask AI to act as a strict embedded systems reviewer. The review should identify missing requirements, unclear interfaces, untested assumptions, weak mitigations, timing concerns, task conflicts, or resource contention.

Use one of these Team Response options: **Fixed**, **Partially fixed**, **Accepted as risk**, **Deferred**, **Rejected with reason**, or **Needs validation**. The action and evidence columns must justify the selected response. **AI reviewer prompt used**	Act as a strict embedded systems reviewer for an autonomous line-following robot car (Raspberry Pi Pico W on a Cytron Robo Pico, µT-Kernel 3.0, C99 / BARR-C:2018) with line following, barcode navigation, hump detection, obstacle avoidance and MQTT telemetry. Check the requirements, pin layout, RTOS task table, task interaction diagram, statecharts, assumptions, and AI evidence. Identify missing requirements, unclear interfaces, untested assumptions, weak safety mitigations, timing risks, task conflicts, and shared-resource problems. Do not rewrite the design; list only gaps that the team must review and justify.

AI-identified gap	Team response	Action taken	Evidence or reason
A STOP press during a sonar scan (2.5–11 s) was missed, STOP in IDLE started a calibration spin, and STOP set DONE without saying the move was cancelled.	Partially fixed	STOP latch set by GP21 IRQ and receive paths before any queue; motion brakes every cycle and returns CANCELLED; vehicle waits sliced ≤ 50 ms; calibration only by command or long START. Code and timing tests pending.	FR12–FR14, NFR3, NFR9, sections 4–5, A14
No watchdog; an indefinite velocity request could keep a car driving if the vehicle task hung; behaviour after reset undefined.	Partially fixed	Velocity requests expire after 100 ms; 250 ms watchdog fed only with vehicle progress < 200 ms; braked start-up, no automatic run. Code and fault injection pending.	FR14, 6.2, A14
usb (3) and cyw43 (4) pre-empt motion (5) and could delay control and STOP.	Needs validation	Measure jitter and STOP latency under load, reconnects and queue flooding; move cyw43 below motion if needed.	NFR1, NFR3, A2, A14
A fixed 60 ms delay was reported as "applied"; queues could drop commands silently or hold a STOP.	Partially fixed	Request ids with accepted / rejected / applied replies on a priority queue with bounded retry and de-duplication; full queues reject all but STOP. Code pending.	FR10, section 4
Barcode: the 250 ms quiet zone delayed commands by 75 mm at 300 mm/s; speed= accepted 400 mm/s; the sensor is faster than cruise in turns; the side flips after a U-turn; 400 mm fallback turn.	Partially fixed	Decode at the 29th element (≈ 9 mm nominal); 300 mm/s limit at the sensor including turns; unused commands discarded; other symbols report-only. Placement and loaded latency: validation planned (A6, A11).	FR3, NFR2, 6.4, A6, A11
Hump height under-read short humps (≈ 29 mm for 40 mm, against ±5 mm) and the 6° trigger missed low humps.	Partially fixed	Front-contact estimator, 10 mm height trigger, history reset on turn / reverse / fault. Simulation is host-only; NFR6 unmet until real humps pass.	6.5, NFR6, A7, A12
HC-SR04 powered at 3.3 V against a 5 V datasheet; raising VCC alone would put 5 V on GP7.	Needs validation	Identify the module and range-test at 3.3 V; fallback is 5 V with a 1 kΩ / 2 kΩ ECHO divider.	Section 2, A9
After an impact the box may sit in the 20 mm blind zone and read as clear; no-echo handling was undefined (unknown treated as clear, or as a stop that would halt the car on open track).	Partially fixed	Reverse 120 mm before scanning after an impact. Pings classed VALID / OUT_OF_RANGE / FAULT: out of range is free space only if A9 confirms the module behaviour; two FAULTs or a stale reading stop. Validation planned (A9).	FR7, NFR5, 6.6, A9
One spurious echo started a scan; FR7 (trigger < 300 mm) conflicted with NFR5 (detect at ≥ 300 mm).	Fixed	Two consecutive readings trigger; NFR5 separates sensing range (400 mm) from the trigger, with a stop budget giving ≈ 190 mm clearance at 300 mm/s.	FR7, NFR5, A3
Bypass legs and reversing were unchecked, and the sonar could be	Partially fixed	Leg check (70° / 90° / 110°) before every forward leg; reverse ≤ 120 mm only on the path	FR8, 6.6, A10

moved between a "centred" check and a ping.		just driven; one semaphore per aim + settle + ping. Front monitor active in all forward travel; near corners and turn sweeps: validation planned (A10).	
Wi-Fi association was never re-joined; only the MQTT session recovered.	Partially fixed	WIFI_REJOIN designed (6.1); implement, then test AP loss and broker loss separately.	FR11, 6.1, A1
Encoder resolution vs the 4 mm completion threshold, direction while coasting, and motor / servo / battery limits were unexamined.	Needs validation	Both edges counted (40 / rev); reversal waits for 250 ms without edges; motion accuracy, braking and current limits measured.	A3, A4, A15, 6.2
IMU configuration, sample freshness, heading wrap-around and magnetic disturbance were undefined; calibration accepted poor IR contrast.	Partially fixed	IMU configuration, freshness checks and fault stop defined; turn rate from encoders cross-checked by magnetometer; calibration rejects < 50 % contrast. Code pending.	FR13, NFR6, 6.5, A5, A16
LINE_SEARCH was not defined anywhere, and "no task blocks on another" ignored the motion and sonar waits.	Fixed	LINE_SEARCH sweeps, limit and failure defined in section 5; section 4 limits the claim to snapshot reads and NFR9 lists every bounded wait.	Sections 4–5, NFR9
GP26 appears on Grove 5 and 6 (Grove 5 also carries GP6); GP28 doubles as VBAT / 2 sense.	Fixed	IR and sonar wired by jumper to the GPIO breakout, AOUT only, pin order checked; VBAT jumper stays open.	Section 2, wiring sketch
Snapshot copies are atomic only on one core; MQTT has no authentication.	Accepted as risk	Single-core profile with a compile-time SMP check; closed lab network and a small parsed command set.	A13, FR10
Repository link empty; build, checker and memory claims unverifiable.	Partially fixed	Repository linked; NFR10 / NFR11 stated as targets with the evidence to attach; RAM defined as linked .data + .bss.	NFR10, NFR11
A spurious OBSTACLE could be raised at run start because the IMU impact counter resets.	Fixed	Only an increase in the impact count counts as an impact.	vehicle.c obstacle_due(); code review

9. Design Consistency

Use this section to demonstrate that the design artefacts describe one coherent system rather than independent diagrams.

Check	Student Response
Which task owns each hardware device?	Motors (GP8–11) and watchdog: motion. Encoders (GP2 / GP3): encoder_edge_isr → motion. IR (GP26–28): ir_sample_isr → vehicle and barcode. IMU (I2C0, GP4 / GP5): imu. HC-SR04 (GP6 / GP7) and servo (GP12): obstacle monitor and vehicle scans under one semaphore; echo_edge_isr times the echo. START (GP20): vehicle. STOP (GP21): stop_btn_isr → stop latch → motion. Wi-Fi: cyw43. USB / UART0: usb and telemetry. LED (GP16): blink.
Which task has the highest priority, and why?	Motion (5) is the highest application task: it owns the motors, runs the 20 ms loop, brakes on the latch and feeds the watchdog. ISRs and the cyclic handler run above it but only record data or signal. The template usb (3) and cyw43 (4) also pre-empt it; their effect is measured (A2, A14).
Which subsystem statechart corresponds to which RTOS task or subsystem?	6.1 MQTT: cyw43 with telemetry. 6.2 Motion: motion. 6.3 Line classifier: vehicle (line_follow.c). 6.4 Barcode: ir_sample_isr and barcode. 6.5 Hump tracker: imu (terrain.c). 6.6 Obstacle: vehicle with obstacle and motion. Section 5 is the vehicle task's state machine; OBSTACLE is expanded in

	6.6, and LINE_SEARCH is defined in section 5 using the 6.3 classifier.
How are functional requirements reflected in the task and statechart design?	FR1–2: vehicle, LINE_FOLLOW / LINE_SEARCH, 6.3. FR3: ir_sample_isr + barcode (6.4), NAV_TURN. FR4: motion (6.2). FR5–6: imu (6.5). FR7–8: OBSTACLE (6.6) with obstacle and motion. FR9–11: telemetry and cyw43 (6.1). FR12–13: stop latch, START checks, STOPPED and CALIBRATE (section 5). FR14: motion watchdog and vehicle progress.
What changes are still expected after the Week 6 review?	Implement the corrections marked "Partially fixed" in 8.3 (they are designed here, not yet in the code), calibrate car_config.h on the car, validate A1–A16 (first the HC-SR04 supply and no-target behaviour, A9), attach the NFR10 / NFR11 evidence, and update these diagrams if integration changes a queue, priority or state.